



AOSTP: Logistics Management System

Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

This document defines the functional and non-functional requirements for the **AOSTP**.

The system will handle end-to-end logistics workflows, including package intake, shipping, invoicing, payment, packing lists, container tracking, and notifications, with strong audit logging and reporting capabilities.

1.2 Scope

The system will be a **web-based application** accessible by authorized company staff (admins, clerks, finance managers) and provide customer-facing access to invoices and shipment information.

Key goals:

- Streamline package intake and tracking.
- Automate invoice generation from packing lists.
- Enable payment processing linked to invoices.
- Provide container and packing list management.
- Maintain strong audit trail and role-based permissions.

1.3 Audience

- **Development team:** for system implementation.
- **Operations team:** for day-to-day system use.
- **Management:** for oversight and decision-making.



2. Actors & Roles

ROLE	DESCRIPTION	KEY PERMISSIONS
SUPER ADMIN	Full system access	All modules, audit logs, configuration
FINANCE MANAGER	Manage exchange rates, approve payments	Exchange rate edits, view/edit invoices
OPERATIONS CLERK	Manage packages, packing lists, containers	Create/edit records, issue invoices
PAYMENT CLERK	Process payments	View invoices, record payments
CUSTOMER	Limited self-service	View invoices, shipment details

3. System-Wide Business Rules

1. **Retention Period:** All business records (packages, invoices, payments, containers, packing lists, audit logs) retained **min. 1 year** (configurable).
2. **Mandatory Photo on Receipts:** Minimum 1 photo required per package at intake.
3. **Customer Contact:** Phone number mandatory for all customers.
4. **Notifications:** Email always sent if available; SMS or WhatsApp sent based on customer preference.
5. **Audit Logging:** All modules log all actions (create, update, delete, status changes) with before/after values. Logs immutable.
6. **Currency & Rates:** Payments use a centrally managed exchange rate (Feature 9).



4. Functional Requirements

Feature 1: Package Intake & Receipt Generation

- Capture package details, assign tracking codes, store photos.
- Manual tracking code default; barcode optional.
- Photo mandatory (min. 1 per receipt).
- Notifications sent via email + preferred channel.
- Retention: min. 1 year.

Feature 2: Customer Management

- Maintain customer records with mandatory phone number.
- Store email, name, ID type & number (reference only).
- Supports linking packages, invoices, and payments.

Feature 3: ID Proof Recording

- Record ID type & number for reference (no image storage).
- Linked to customer profile for verification.

Feature 4: Shipping Methods & Rate Calculations

- Modes: **Sea (CBM)** and **Air (Weight)**.
- Air subtypes: Normal Air, Express Air, Battery Goods, Phones (different rates).
- Admin defines rates per mode/subtype.

Feature 5: Payment Management

- Search customer, select pending invoices, apply exchange rate, process payment.
- Supports payment for one or multiple invoices.
- Generates receipt with container date or pickup codes.
- Updates package payment status automatically.

Feature 6: Warehouse Management

- Track packages in warehouse (location, days in storage).



Feature 7: Notifications Management

- Email always sent if email exists.
- SMS or WhatsApp sent based on customer preference.
- Triggers: package intake, invoice issued, payment received, shipment updates.

Feature 8: Packing List Management

- Group shipments per container with details: loading date, city, ETA (40–45 days).
- Unique list name (default = loading date).
- Exportable to PDF/Excel.
- Packages must be linked to a single packing list at a time.

Feature 9: Exchange Rate Management

- Only Finance Manager/System Admin can set/edit USD > Local rate.
- Single active rate per currency pair.
- Historical rates stored with audit log.

Feature 10: Container Management

- Maintain container records (number, loading date, destination).
- Status flow: planned > loaded > shipped > arrived > closed.
- Containers linked to packing lists.

Feature 11: Invoice Management

- Auto-generated from packing lists on save (one invoice per customer).
- Includes package details, CBM, rates, amounts.
- Posted to customer profile immediately.
- No manual invoices.

Feature 12: Audit Log Management

- Logs all actions in all modules with before/after values.



- Super Admin can view and export logs.
- Logs immutable, retained min. 1 year.

5. Non-Functional Requirements

- **Performance:** Must handle 100+ concurrent users and large packing lists.
- **Security:** Role-based access, 2FA, encryption for data in transit.
- **Compliance:** GDPR-like data minimization (only store necessary personal info).
- **Availability:** 99% uptime target.
- **Scalability:** Design must allow adding new shipment modes, notification channels.

6. Implementation & Technical Requirements

6.1 Overview

The API backend shall follow a modular, feature-first architecture. Each major business feature described in this SRS (for example: Package Intake, Packing List, Container Management, Payment, Notification) must be implemented as a self-contained module. Prisma shall be used as the ORM to manage database schema and migrations. Data Transfer Objects (DTOs) shall be defined for all API inputs and outputs. JWT-based authentication with role-based authorization shall be implemented. Unit tests must cover services and critical modules. Ensure best practices for input sanitization, environment variable validation, and structured logging.

6.2 Required Technology Choice

- Node.js (LTS) with TypeScript.
- NestJS framework using a module-per-feature organization.
- Prisma for database access and migrations.
- JWT for authentication (access + refresh tokens) and role-based authorization.
- AWS S3 for object storage (only S3; no alternative providers).
- Pluggable provider modules for SMS/notifications and payments (providers added when required).
- Structured logging and error tracking (production-ready sink).



- Monitoring and alerting (metrics and error reporting).

6.3 Project & Module Organization

- One NestJS module per feature. Each module must include its own controllers, services, DTO definitions, and data access (through Prisma).
- Controllers are thin: map requests to DTOs and call services. Business logic belongs in services.
- Common shared elements (global guards, interceptors, filters, DTO utilities, correlation id middleware, and Prisma service) are placed under a shared/common area.
- Notifications and Payments must be implemented as provider-agnostic modules with a defined interface so providers can be plugged in or replaced without changing business logic.

6.4 API & Module Rules

- DTOs and validation must be used for all requests. The API must reject unexpected fields and validate types and constraints.
- Inputs must be sanitized (trim strings, remove or escape HTML where required, validate file types and sizes).
- Prisma must be accessed through a single Prisma service to centralize connection and transaction handling.
- Use database transactions for multi-step operations that must be atomic (for example, invoice generation and posting).
- Implement API versioning to support future changes (for example: /v1/...).
- API documentation shall be available (auto-generated OpenAPI/Swagger for developer use, with production access restricted).

6.5 Authentication & Authorization

- JWT authentication with short-lived access tokens and longer-lived refresh tokens. There must be a revocation mechanism for refresh tokens.
- Role-based access control: endpoints must declare required roles and the system must enforce them consistently.
- Optional 2FA support must be available for users who enable it.
- Sensitive operations (for example: payment release, refunds, administrative user changes) must require appropriate role checks and, where appropriate, additional verification steps.

6.6 Object Storage (AWS S3) — Requirements

- AWS S3 is the only approved object-store provider for the system. All files (images, receipts, generated documents) must be stored in S3.



- Use signed/presigned URLs for client uploads to minimize server-side bandwidth and to reduce exposure of credentials.
- All buckets must enforce server-side encryption and follow a least-privilege IAM policy model.
- Versioning and appropriate lifecycle rules (archival to cost-effective storage tiers) must be configured in line with retention policies.
- Credentials for S3 access must never be stored in the code repository; use instance roles, secrets manager, or similar secure mechanisms.

6.7 Notifications, SMS & Payments (Pluggable)

- Notifications (SMS, email, WhatsApp, push) must be implemented behind a provider-agnostic interface so providers can be added later without code changes to business logic.
- SMS and Payment provider modules may be added at deployment time. The system must function with or without a provider configured (fallback behavior, such as email or logged notifications, must be defined).
- Provider configuration must be environment-driven (API keys, sender IDs, regions) and validated at startup.

6.8 Logging, Observability & Correlation

- The system shall produce structured logs (JSON) that include key metadata: timestamp, severity, service/module, request/correlation id, user id (when available), and trace identifiers.
- Exception/Crash monitoring must be configured (error tracking service) to capture uncaught exceptions and stack traces.
- Access logs shall capture request method, path, response status, latency, and user identification where available.
- A correlation id must be generated per incoming request and propagated through logs, downstream calls, and any asynchronous jobs related to the request.

6.9 Environment & Configuration Management

- Environment variables must be validated at service startup against a defined schema; the application must fail fast if required configuration is missing or malformed.
- Provide a clear env.example file listing required variables and acceptable formats.
- For production, secrets must be stored and accessed via secure secret management (for example, AWS Secrets Manager) and never persisted in source control or plain text files.



6.10 Input Sanitization & Security Hardening

- Enforce strict input validation and sanitization for all API endpoints.
- Apply standard web security middleware and controls: HTTP security headers, rate-limiting, and prevention of parameter pollution.
- CORS must be configured with a strict allowlist of origins.
- File uploads must be validated for type and size and scanned if applicable.
- Use secure password storage (modern hashing algorithms) and avoid storing sensitive tokens unencrypted.
- Avoid unsafe raw queries; when necessary, ensure parameterization and validation.

6.11 Error Handling & Response Consistency

- Define and adopt a consistent API error response schema for all endpoints (status code, standard error message, and structured details when needed).
- Centralized error handling must convert internal errors and database errors into appropriate HTTP responses and log useful diagnostic information for incident investigation.

6.12 Testing Strategy

- Unit tests must cover service-level business logic and critical validation paths.
- Integration tests shall exercise key flows against a test database instance using migrations and test data.